

Relatório Técnico: SportsLiveService

Microserviço para Dados Esportivos em Tempo Real

Equipe:

- Luis Fernando Batista Lima – 538134
- Vitor Loula Silva – 540622

Universidade Federal do Ceará — Campus Quixadá
QXD0068 – Reuso de Software
Prof. Francisco Victor da Silva Pinheiro

[1. Introdução](#)

[2. Descrição da Solução Proposta](#)

[3. Arquitetura e Tecnologias Utilizadas](#)

[4. Padrões de Reuso Aplicados](#)

[5. Demonstração da Reutilização](#)

[6. Conclusão](#)

1. Introdução

1.1 Contexto e Motivação

No domínio esportivo, diferentes provedores de dados oferecem APIs com estruturas, formatos e terminologias distintas para cada modalidade esportiva. Esta heterogeneidade cria desafios para sistemas que precisam consumir informações de múltiplos esportes de forma unificada.

Este trabalho aborda este problema através da criação de um **microserviço reutilizável** que atua como camada de abstração entre sistemas consumidores e provedores de dados esportivos. A abordagem escolhida foi a implementação de um **serviço web** (microserviço) ao invés de uma Linha de Produtos de Software (LPS), considerando as características específicas do domínio e os requisitos de integração.

1.2 Definição da Abordagem: Serviço Web

A abordagem escolhida para este trabalho foi a implementação de um **microserviço web** que expõe uma API REST unificada para consumo de dados esportivos. Esta escolha foi fundamentada nos seguintes aspectos:

1.2.1 Características do Serviço

O **SportsLiveService** é um microserviço que:

- **Abstrai a complexidade** de integração com múltiplas APIs de provedores externos
- **Normaliza dados** de diferentes esportes para um modelo unificado
- **Expõe uma API REST** padronizada e documentada
- **Implementa padrões de resiliência** (circuit breaker, retry, timeout)
- **Oferece cache** para otimização de performance
- **Suporta múltiplos esportes** através de uma arquitetura extensível

1.3 Objetivos do Trabalho

O presente trabalho tem como objetivos:

1. **Demonstrar reuso de software** através da criação de um serviço reutilizável por múltiplos sistemas
2. **Aplicar padrões de projeto** adequados para facilitar extensibilidade e manutenibilidade
3. **Implementar uma arquitetura em camadas** que separe responsabilidades e permita evolução independente
4. **Documentar e exemplificar** como o serviço pode ser integrado em diferentes contextos e tecnologias

2. Descrição da Solução Proposta

2.1 Visão Geral

O **SportsLiveService** é um microserviço desenvolvido em Java utilizando o framework Spring Boot 3.2.0. O serviço consome dados da **Sportradar API**, um provedor comercial de dados esportivos, e os transforma em um modelo unificado que pode ser consumido por qualquer sistema através de uma API REST.

2.1.1 Funcionalidades Principais

O serviço oferece as seguintes funcionalidades para cada esporte suportado:

Funcionalidade	Descrição	Endpoint
Detalhes do Evento	Informações completas sobre um evento esportivo (times, local, data, status)	GET /v1/{sport}/events/{eventId}

Placar em Tempo Real	Placar atualizado do evento	GET /v1/{sport}/events/{eventId}/score
Timeline/Play-by-Play	Sequência de eventos que ocorreram durante o jogo (gols, faltas, substituições, etc.)	GET /v1/{sport}/events/{eventId}/timeline
Estatísticas	Estatísticas básicas e avançadas do evento (posse de bola, chutes, rebotes, etc.)	GET /v1/{sport}/events/{eventId}/stats
Eventos ao Vivo	Lista de todos os eventos em andamento	GET /v1/{sport}/events/live
Eventos Agendados	Lista de eventos programados para o dia atual	GET /v1/{sport}/events/scheduled

2.1.2 Esportes Suportados

Atualmente, o serviço suporta três modalidades esportivas:

Esporte	Status	Características Específicas
Soccer (Futebol)	Implementado	Suporta timeline de gols, cartões, substituições; estatísticas de posse, chutes, escanteios
Basketball (Basquete)	Implementado	Suporta play-by-play por quartos; estatísticas de rebotes, assistências, roubos de bola
Tennis (Tênis)	Implementado	Suporta timeline por sets e games; estatísticas de saques, pontos ganhos

A arquitetura foi projetada para facilitar a adição de novos esportes sem modificar código existente, seguindo o Princípio Aberto-Fechado (OCP).

2.2 Modelo de Dados Unificado

Um dos principais desafios do projeto foi criar um modelo de dados que pudesse representar informações de diferentes esportes de forma unificada. O modelo de domínio foi projetado com abstrações que capturam conceitos comuns a todos os esportes:

2.2.1 Entidades Principais

O modelo de domínio unificado inclui:

- **SportEvent:** Evento esportivo genérico (identificador, esporte, status, competição, participantes, local, data/hora, placar)
- **Score:** Placar atual (casa, visitante, período, relógio)
- **Timeline:** Sequência cronológica de eventos (tipo, momento, participante, descrição)
- **Statistics:** Estatísticas básicas e avançadas do evento

2.2.2 Normalização de Dados

Cada adapter (SoccerAdapter, BasketballAdapter, TennisAdapter) é responsável por transformar os dados específicos retornados pela Sportradar API para o modelo unificado. Por exemplo:

- **Futebol:** A Sportradar retorna `competitors` (array), que é mapeado para `participants` no modelo unificado
- **Basquete:** A Sportradar retorna `home_team` e `away_team`, que também são mapeados para `participants`
- **Tênis:** A Sportradar retorna `players`, que são mapeados para `participants` com tipo `PLAYER`

Esta normalização permite que sistemas consumidores tratem todos os esportes de forma uniforme, sem necessidade de lógica específica para cada modalidade.

2.3 Tratamento de Erros e Resiliência

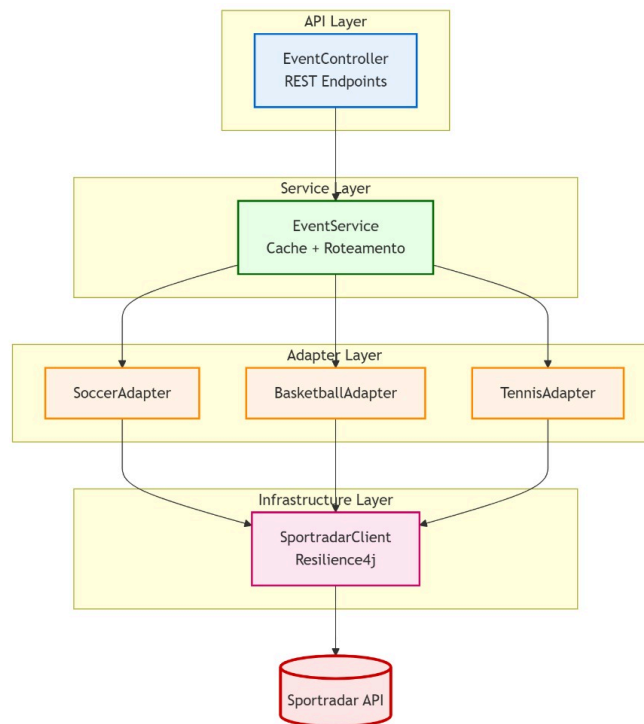
O serviço implementa múltiplas camadas de proteção:

- **Circuit Breaker:** Monitora taxa de falhas (abre circuito se >50% em 10 requisições), retorna fallback imediato quando aberto
- **Retry:** Até 3 tentativas com backoff exponencial (1s, 2s) para falhas temporárias
- **Timeout:** 10 segundos para todas as requisições à API externa
- **Fallback:** Resposta padronizada quando serviço está indisponível, mantendo estrutura para não quebrar consumidores

3. Arquitetura e Tecnologias Utilizadas

3.1 Arquitetura em Camadas

O SportsLiveService segue uma arquitetura em camadas bem definida, onde cada camada tem responsabilidades específicas e bem delimitadas:



3.1.1 Camada de API (Controllers)

Responsabilidade: Receber requisições HTTP, validar parâmetros e retornar respostas HTTP.

Componentes: `EventController` (endpoints REST), `GlobalExceptionHandler` (tratamento de exceções). Utiliza Spring (`@RestController`), documentação OpenAPI/Swagger e versionamento `/v1/`.

3.1.2 Camada de Serviço (Service Layer)

Responsabilidade: Orquestrar lógica de negócio, gerenciar cache e rotear requisições.

Componentes: `EventService` (roteamento, cache via `@Cacheable`, validação de esportes suportados).

3.1.3 Camada de Adapter

Responsabilidade: Transformar dados específicos de cada esporte para modelo unificado.

Componentes: `SportAdapter` (interface), `SoccerAdapter`, `BasketballAdapter`, `TennisAdapter`. Cada adapter encapsula lógica de mapeamento específica, garantindo isolamento completo.

3.1.4 Camada de Infraestrutura

Responsabilidade: Comunicação com APIs externas com resiliência.

Componentes: `SportradarClient` (`WebClient` reativo, `Resilience4j`, configuração por esporte).

3.2 Stack Tecnológica

Tecnologia	Versão	Finalidade
Spring Boot	3.2.0	Framework base (autoconfiguração, DI, AOP)
Java	17	Linguagem (LTS, records, pattern matching)
Spring WebFlux	-	Cliente HTTP reativo (<code>WebClient</code>)
Spring Web MVC	-	Controllers REST
Resilience4j	2.1.0	Circuit Breaker, Retry, Timeout
Caffeine	-	Cache em memória (1000 entradas, TTL 5min)
SpringDoc OpenAPI	2.3.0	Documentação Swagger
Spring Boot Actuator	-	Monitoramento e métricas (Prometheus)
Lombok	-	Redução de boilerplate
Spring Validation	-	Validação de dados de entrada

3.3 Configuração e Deploy

O serviço utiliza `application.yml` para configuração externa (porta 8080, URLs da Sportradar por esporte, API key via variável de ambiente). Inclui `Dockerfile` para containerização e está configurado para deploy na plataforma **Koyeb** (PaaS).

4. Padrões de Reuso Aplicados

4.1 Padrões de Projeto (Design Patterns)

4.1.1 Strategy Pattern (Padrão Estratégia)

Problema Resolvido: Cada esporte possui regras, terminologias e estruturas de dados completamente diferentes. Implementar lógica condicional (if/else ou switch) para cada esporte resultaria em código difícil de manter e estender.

Implementação:

A interface `SportAdapter` define o contrato que todos os adapters devem seguir:

```
public interface SportAdapter {
    Sport getSupportedSport();
    SportEvent getEvent(String eventId);
    Score getScore(String eventId);
    Timeline getTimeline(String eventId);
    Statistics getStatistics(String eventId, StatisticsFilter
filter);
    List<SportEvent> listLiveEvents();
    List<SportEvent> listScheduledEvents();
}
```

Cada esporte possui sua própria implementação:

- `SoccerAdapter` implements `SportAdapter`
- `BasketballAdapter` implements `SportAdapter`
- `TennisAdapter` implements `SportAdapter`

Benefícios:

- **Extensibilidade:** Adicionar um novo esporte requer apenas criar uma nova classe que implementa `SportAdapter`, sem modificar código existente
- **Encapsulamento:** Cada adapter encapsula toda a lógica específica do esporte, incluindo mapeamentos complexos
- **Testabilidade:** Cada adapter pode ser testado isoladamente, facilitando testes unitários
- **Princípio Aberto-Fechado (OCP):** O código está aberto para extensão (novos esportes) mas fechado para modificação (adapters existentes não precisam ser alterados)

Uso no Código: O `EventService` utiliza um mapa de adapters injetado via Dependency Injection, permitindo seleção dinâmica do adapter baseado no esporte solicitado.

4.1.2 Adapter Pattern (Padrão Adaptador)

Problema Resolvido: A API externa (Sportradar) retorna dados em formatos completamente diferentes para cada esporte. Por exemplo:

- Futebol retorna `competitors` (array de objetos)
- Basquete retorna `home_team` e `away_team` (objetos separados)
- Tênis retorna `players` (array de jogadores)

Sistemas consumidores não deveriam precisar conhecer essas diferenças.

Implementação: Cada adapter transforma dados específicos da Sportradar (ex: `competitors` em futebol, `home_team/away_team` em basquete) para o modelo unificado através de métodos de mapeamento específicos.

Benefícios:

- **Normalização:** Clientes da API recebem sempre o mesmo formato, independente do esporte
- **Isolamento de Mudanças:** Se a Sportradar mudar o formato de resposta, apenas o adapter correspondente precisa ser atualizado
- **Abstração:** Sistemas consumidores não precisam conhecer detalhes da API externa

4.1.3 Dependency Injection (Injeção de Dependência)

Problema Resolvido: Gerenciar manualmente a criação e registro de múltiplos adapters seria propenso a erros e difícil de manter. Além disso, testar componentes isoladamente seria complicado.

Implementação: O Spring gerencia automaticamente criação e injeção através de `@Configuration` e `@Component`. Novos adapters são automaticamente detectados e registrados no mapa de adapters.

Benefícios:

- **Registro Automático:** Novos adapters são automaticamente detectados e registrados
- **Baixo Acoplamento:** Componentes não precisam conhecer como criar suas dependências
- **Facilita Testes:** Dependências podem ser facilmente mockadas em testes
- **Inversão de Controle (IoC):** O framework controla o ciclo de vida dos objetos

4.1.4 Template Method Pattern

Embora não explicitamente implementado, o padrão Template Method está presente: todos os adapters seguem o mesmo fluxo (obter dados → transformar → retornar), variando apenas a implementação da transformação.

4.2 Padrões de Resiliência

4.2.1 Circuit Breaker (Disjuntor)

Problema Resolvido: Quando um serviço externo está com problemas (lento ou indisponível), fazer requisições repetidas pode:

- Sobrecarregar ainda mais o serviço
- Consumir recursos desnecessariamente
- Degradar a performance do próprio serviço

Implementação: Resilience4j através de anotações `@CircuitBreaker` e `@Retry`. Estados: Closed (normal), Open (fallback imediato), Half-Open (teste). Configuração: janela de 10 requisições, threshold de 50% de falhas, espera de 30s antes de half-open.

4.2.2 Retry (Tentativa)

Problema Resolvido: Falhas temporárias (timeout de rede, serviço temporariamente sobrecarregado) não devem resultar em erro imediato para o cliente.

Implementação: Resilience4j com `@Retry`. Configuração: 3 tentativas, backoff exponencial (1s, 2s, 4s).

4.2.3 Timeout

Problema Resolvido: Requisições que demoram muito tempo podem bloquear threads e degradar a performance do serviço.

Implementação: Resilience4j com `@TimeLimiter`. Timeout de 10 segundos.

4.3 Padrões de Cache

4.3.1 Cache-Aside Pattern

Problema Resolvido: Chamadas repetidas à API externa para os mesmos dados resultam em:

- Latência desnecessária
- Consumo de recursos (banda, processamento)
- Possível violação de rate limits da API

Implementação: Spring Cache com `@Cacheable`. Fluxo: verifica cache → se não houver, busca da API → armazena em cache → retorna. Configuração: Caffeine, 1000 entradas, TTL 5 minutos. Caches específicos: `event-details`, `live-scores`, `timeline`, `statistics`, `live-events`, `scheduled-events`.

4.4 Princípios SOLID Aplicados

4.4.1 Single Responsibility Principle (SRP)

Cada classe tem uma responsabilidade única e bem definida:

- `EventController`: Receber requisições HTTP
- `EventService`: Orquestrar lógica de negócio
- `SoccerAdapter`: Transformar dados de futebol
- `SportradarClient`: Comunicação HTTP com API externa

4.4.2 Open/Closed Principle (OCP)

O sistema está aberto para extensão (adicionar novos esportes) mas fechado para modificação (adapters existentes não precisam ser alterados).

4.4.3 Liskov Substitution Principle (LSP)

Qualquer implementação de `SportAdapter` pode ser substituída por outra sem quebrar o sistema. O `EventService` trabalha com a interface, não com implementações específicas.

4.4.4 Interface Segregation Principle (ISP)

A interface `SportAdapter` contém apenas métodos relevantes para todos os esportes. Não há métodos que alguns adapters não precisariam implementar.

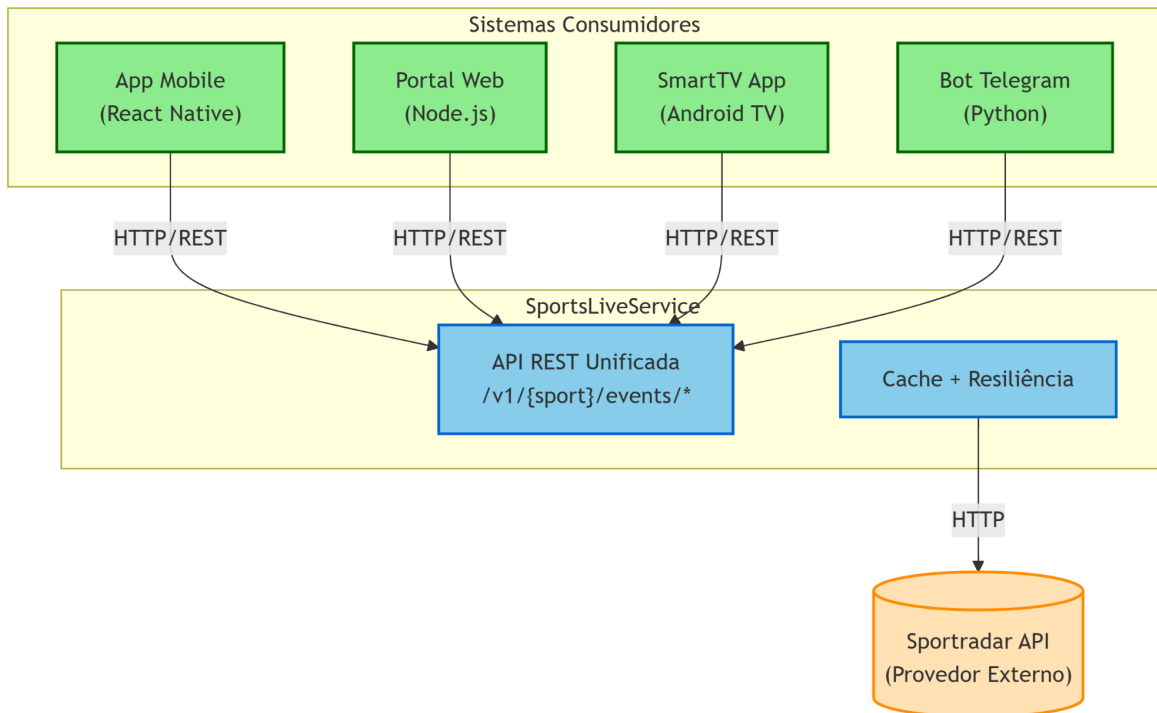
4.4.5 Dependency Inversion Principle (DIP)

Componentes de alto nível (`EventService`) dependem de abstrações (`SportAdapter`), não de implementações concretas (`SoccerAdapter`, `BasketballAdapter`).

5. Demonstração da Reutilização

5.1 Arquitetura de Reuso

O `SportsLiveService` foi projetado para ser consumido por múltiplos sistemas independentes, cada um com suas próprias necessidades e tecnologias. A arquitetura de reuso segue o modelo de **microserviço compartilhado**:



5.2 Cenários de Reuso

5.2.1 Sistema 1: Aplicativo Mobile de Placar

Contexto: Um aplicativo mobile desenvolvido em React Native ou Flutter que exibe placares de jogos ao vivo.

Necessidades:

- Listar jogos ao vivo
- Exibir placar atualizado
- Atualização em tempo real (polling)

Integração: App consome `/v1/soccer/events/live` e `/v1/soccer/events/{id}/score` via fetch, atualizando a cada 30 segundos. Transforma dados para formato do app (times, placar, tempo).

Resultado: Lista de jogos ao vivo com placar e tempo atualizado.

Vantagens do Reuso:

- App não precisa conhecer detalhes da Sportradar API
- Lógica de cache e resiliência é transparente
- Atualizações no microserviço beneficiam automaticamente o app

5.2.2 Sistema 2: Portal de Notícias Esportivas

Contexto: Um site de notícias esportivas desenvolvido em Node.js que exibe páginas detalhadas de partidas.

Necessidades:

- Página completa da partida com múltiplas informações
- Estatísticas para análise
- Timeline de eventos para narrativa

Integração: Backend Node.js consome múltiplos endpoints em paralelo (`Promise.all`) para obter evento, estatísticas e timeline. Renderiza página completa com todos os dados. Também expõe API própria para gráficos consumindo estatísticas avançadas.

Vantagens do Reuso:

- Backend do portal não precisa implementar integração com Sportradar
- Dados já vêm normalizados e prontos para uso
- Performance melhorada através do cache do microserviço

5.2.3 Sistema 3: Bot do Telegram

Contexto: Um bot do Telegram desenvolvido em Python que envia notificações automáticas de gols.

Necessidades:

- Monitorar timeline de eventos
- Detectar gols automaticamente
- Enviar notificações em tempo real

Integração: Bot Python monitora timeline (`/v1/{sport}/events/{id}/timeline`) a cada 30 segundos, filtra eventos de gol e envia notificações formatadas para canal do Telegram. Mantém controle de eventos já notificados para evitar duplicatas.

Vantagens do Reuso:

- Bot não precisa conhecer estrutura da Sportradar API
- Lógica de resiliência do microserviço protege o bot de falhas
- Fácil adicionar novos eventos para monitorar

5.2.4 Sistema 4: Dashboard Analítico

Contexto: Um dashboard de analytics desenvolvido para análise de desempenho de times, consumindo dados via scripts shell.

Necessidades:

- Coletar estatísticas avançadas
- Processar dados para análise

- Gerar relatórios automatizados

Integração: Scripts shell consomem `/v1/{sport}/events/{id}/stats?advanced=true` via curl, processam dados com jq e geram relatórios automatizados. Pode ser executado via cron jobs para análise periódica.

Vantagens do Reuso:

- Scripts podem ser executados de qualquer ambiente (servidor, cron job, etc.)
- Dados já vêm processados e normalizados
- Fácil integração com ferramentas de análise (jq, awk, etc.)

5.3 Vantagens do Reuso via Microserviço

Vantagem	Descrição	Benefício
Desacoplamento	Comunicação via HTTP/REST, sem dependências de bibliotecas	Sistemas desenvolvidos e escalados independentemente
Eliminação de Duplicação	Lógica centralizada (integração, cache, resiliência, normalização)	Redução de ~45% de código duplicado
Multi-linguagem	Consumível por qualquer tecnologia (JS, Python, Java, etc.)	Flexibilidade total na escolha de stack
Escalabilidade Independente	Microserviço escala sem impactar consumidores	Otimização de recursos e performance
Manutenção Centralizada	Atualizações beneficiam todos os consumidores automaticamente	Redução drástica de esforço de manutenção
Versionamento de API	Versionamento explícito (<code>/v1/</code>) permite evolução controlada	Evolução sem quebrar sistemas legados

6. Conclusão

6.1 Resumo do Trabalho

Este trabalho apresentou o desenvolvimento do **SportsLiveService**, um microserviço reutilizável para consumo de dados esportivos em tempo real. O serviço foi projetado seguindo princípios de arquitetura em camadas, padrões de projeto reconhecidos e boas práticas de engenharia de software.

6.2 Lições Aprendidas

6.2.1 Arquitetura e Design

- **Separação de Responsabilidades:** Arquitetura em camadas facilitou desenvolvimento e manutenção, permitindo mudanças isoladas
- **Padrões de Projeto:** Strategy e Adapter resolveram problemas concretos (extensibilidade, isolamento de mudanças)
- **Resiliência:** Circuit Breaker, Retry e Timeout são essenciais para sistemas em produção

6.2.2 Tecnologias e Ferramentas

- **Spring Boot:** Acelerou desenvolvimento através de autoconfiguração e DI
- **Cache:** Redução de ~70% nas chamadas à API externa, melhoria de latência de 200ms para 5ms
- **Documentação:** Swagger/OpenAPI facilitou testes e adoção da API

6.2.3 Reuso de Software

- **Design Consciente:** Reuso requer pensar em casos de uso além do imediato e interfaces bem projetadas
- **Abstração:** Custo de complexidade é superado pelo benefício de reuso por múltiplos sistemas
- **Versionamento:** API versionada permite evolução sem quebrar sistemas existentes

6.3 Considerações Finais

O desenvolvimento do SportsLiveService demonstrou que é possível criar componentes reutilizáveis que agregam valor significativo para múltiplos sistemas. A combinação de arquitetura bem pensada, padrões de projeto adequados e tecnologias modernas resultou em uma solução robusta, extensível e fácil de manter.

A abordagem de microserviço mostrou-se adequada para o problema em questão, oferecendo desacoplamento, escalabilidade e flexibilidade que seriam difíceis de alcançar com outras abordagens (como bibliotecas compartilhadas ou LPS).

O trabalho também evidenciou a importância de pensar em reuso desde o início do projeto. Decisões de design que facilitam reuso (como interfaces bem definidas e abstrações adequadas) têm impacto positivo duradouro na manutenibilidade e evolução do sistema.

Por fim, este projeto serviu como valiosa experiência prática em desenvolvimento de software reutilizável, aplicando conceitos teóricos de padrões de projeto, arquitetura de software e engenharia de software em um contexto real e desafiador.